Homework Assignment 2: Data-Based (Surrogate) Modeling

- **Learning outcome:** By completing this assignment, you will be able to design a sampling plan for building a data-based model; fit polynomial-regression, radial-basis-function, Kriging, and neural-network surrogates; recognize that every surrogate family has a *complexity knob* trading bias for variance, and select that knob by cross-validation error rather than training error; and diagnose and remedy ill-conditioning in least-squares and Kriging.
- You may choose to use either programming language (MATLAB or Python) for the questions where coding is required. MATLAB users should submit Live Scripts (use *Publish* to generate a PDF of your code); Python users should submit Jupyter notebooks with all cells executed and saved.
- Your submission should include: (i) a PDF report named `<McGill_ID>_HW2.pdf` explaining your solution approach, and (ii) either a published MATLAB script or a Python notebook named `<McGill_ID>_HW2.ipynb` or `<McGill_ID>_HW2.m`, respectively.

Dataset

This assignment uses a physics-generated engineering dataset provided in the `Datasets/` folder of the course repository: `aero_naca4_airfoil_panel.csv` – a NACA 4-digit airfoil evaluated with a linear-strength vortex-panel method (Prandtl–Glauert compressibility correction, engineering viscous-drag estimate), $p = 500$ samples, sampled with a Latin hypercube design and split ∼80/20 `train`/`test` (column `split`). **Inputs (6):** `max_camber_m`, `camber_pos_p`, `thickness_t`, `alpha_deg`, `mach`, `reynolds`. **Output used throughout:** `CL` (lift coefficient). *Note:* `reynolds` spans nearly two orders of magnitude (2×10^5–10^7); for every model in questions 2–6, use $\log_{10}$(`reynolds`) in place of the raw column as your sixth input feature, and keep the other five inputs on their natural scale unless a question tells you to rescale them. Wherever this assignment says "scaled inputs", use per-column min–max scaling to $[0, 1]$ with bounds computed from the `train` rows (apply the same bounds to the `test` rows). *Note:* p always denotes a number of samples, never the input `camber_pos_p`.

The training set is small ($p \approx 400$) and should always be used in full.

Software

For this assignment you will use:

- **NumPy/SciPy/scikit-learn** (Python) or the **Statistics and Machine Learning Toolbox** (MATLAB); no other library is needed.
- **Starter code.** You do *not* need to write every model from a blank page: Lecture 4's companion notebooks already contain working reference implementations of every method below (on smaller toy examples). Adapt/generalize this code to the datasets and questions below rather than re-deriving it:

  - `L04_polynomial_regression.ipynb` – design matrix, normal-equations fit, condition number, min-max scaling, ridge fit, λ-vs-RMSE sweep. Used in questions 2–3.
  - `L04_radial_basis_functions.ipynb` – RBF fit/predict and kernel functions. Used in question 4.
  - `L04_kriging_gaussian_process.ipynb` – correlation matrix, Kriging fit/predict (nugget built in). Used in question 6.
  - `L04_surrogate_model_comparison.ipynb` – a ready-made k-fold CV utility plus unified fit/predict wrappers for ridge, RBF, and Kriging. Reusable throughout questions 3–6 and the bonus comparison question.
  - `L04_design_of_experiments.ipynb` and `L04_neural_networks.ipynb` are useful references too, though question 5 is easiest solved directly with the library function.

- **Assignment templates.** A skeleton file with the imports/data loading done for you and a `TODO`/hint block for each question is provided for both languages – use it as a starting point rather than setting up the problem from scratch: `HW2_template.ipynb` (Python) and `HW2_template.m` (MATLAB).

Questions

1. **Design of experiments.** `aero_naca4_airfoil_panel.csv` was sampled with LHS ($n = 6$, $p = 500$), not a full-factorial grid. An LHS with p points has p distinct levels in every 1-D projection. Compute the full-factorial grid size needed to match this 1-D resolution (p^n runs) and compare it to p; conversely, find how many levels per dimension a grid of at most p runs can afford ($\lfloor p^{1/n} \rfloor$). Briefly explain why LHS avoids this *curse of dimensionality* while still space-filling well in 1-D.

2. **Polynomial regression from first principles.** Using the `train` rows, fit an affine model $\hat{y}(\mathbf{x}; \mathbf{w}) = w_0 + \sum_{j=1}^{6} w_j x_j$ for `CL` (inputs: `max_camber_m`, `camber_pos_p`, `thickness_t`, `alpha_deg`, `mach`, $\log_{10}$(`reynolds`)) by solving the normal equations $\mathbf{Z}^\mathsf{T}\mathbf{Z}\,\mathbf{w} = \mathbf{Z}^\mathsf{T}\mathbf{y}$ directly – adapt the Starter Code notebook's `normal_equations_fit`, or assemble $\mathbf{Z}$ yourself and pass $\mathbf{Z}^\mathsf{T}\mathbf{Z}$ and $\mathbf{Z}^\mathsf{T}\mathbf{y}$ to `numpy.linalg.solve` or MATLAB's backslash operator (no canned regression function; note that `lstsq`, or backslash applied to $\mathbf{Z}$ and $\mathbf{y}$ directly, does *not* form $\mathbf{Z}^\mathsf{T}\mathbf{Z}$). Report $\mathbf{w}$ and the test RMSE/R^2, then confirm both match `scikit-learn`'s `LinearRegression` (or MATLAB's `fitlm`).

3. **Ill-conditioning and ridge regularization.** Extend question 2 to a pure quadratic (add $w_{j+6}\, x_j^2$, 13 columns in $\mathbf{Z}$) and report $\kappa(\mathbf{Z}^\mathsf{T}\mathbf{Z})$. Min-max scale the inputs to $[0, 1]$ (training-set bounds), rebuild $\mathbf{Z}$, and report the new condition number. Sweep the ridge penalty $\lambda \in \{0, 10^{-4}, 10^{-2}, 1, 100\}$ (adapt `ridge_fit`, i.e. $\mathbf{w}_{\text{ridge}} = (\mathbf{Z}^\mathsf{T}\mathbf{Z} + \lambda\mathbf{I})^{-1}\mathbf{Z}^\mathsf{T}\mathbf{y}$ on the scaled $\mathbf{Z}$; this penalizes w_0 too, so if you use `sklearn.linear_model.Ridge` pass your own column of ones with `fit_intercept=False` to get the same result); using 5-fold CV on the training set, plot CV RMSE vs. λ, report the best λ, and compare its test RMSE/R^2 to $\lambda = 0$. Relate this to the bias–variance trade-off and to $\kappa(\mathbf{Z}^\mathsf{T}\mathbf{Z} + \lambda\mathbf{I})$.

4. **RBF and its spread parameter.** Fit a Gaussian RBF surrogate $\phi(r) = \exp(-\lambda r^2)$ for `CL` (same 6 scaled inputs) with the Starter Code notebook (replace its 1-D distance by the Euclidean distance in 6-D) or a library, sweeping the spread over ≥ 4 values across orders of magnitude (e.g. $\lambda \in [10^{-4}, 10^{3}]$; large λ = narrow basis functions). If you use `scipy.interpolate.RBFInterpolator`, set `kernel='gaussian'` and `epsilon`= $\sqrt{\lambda}$ – the default thin-plate kernel ignores `epsilon`; MATLAB's `newrbe` spread s corresponds to $\lambda = (0.8326/s)^2$. Using 5-fold CV, plot CV RMSE vs. spread and identify the best value. Relate the too-narrow/too-wide extremes to the "good spread" vs. "poor spread" behavior from lecture.

5. **Neural network width as a complexity knob.** Fit a single-hidden-layer network for `CL` (same 6 scaled inputs) with `MLPRegressor` (use `solver='lbfgs'`, `max_iter=5000`, a fixed `random_state`; the default `adam` with 200 iterations does not converge here) or `fitrnet`, sweeping the hidden-layer width over ≥ 4 values (e.g. 2, 8, 32, 64). Using 5-fold CV, plot CV RMSE vs. width and identify where under- and overfitting set in.

6. **Kriging: correlation-matrix conditioning and uncertainty.** Draw a fixed random subsample of $p = 25$ training points (e.g. `np.random.default_rng(559)`) and use scaled inputs throughout this question. Using $R(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\theta\|\mathbf{x}_i - \mathbf{x}_j\|^2)$ and the ordinary-Kriging equations from lecture ($\hat{\beta} = (\mathbf{1}^\mathsf{T}\mathbf{R}^{-1}\mathbf{1})^{-1}\mathbf{1}^\mathsf{T}\mathbf{R}^{-1}\mathbf{y}$, $\hat{y}(\mathbf{x}_{\text{new}}) = \hat{\beta} + \mathbf{r}^\mathsf{T}\mathbf{R}^{-1}(\mathbf{y} - \hat{\beta}\mathbf{1})$), adapt the Starter Code notebook's Kriging fit/predict (not a black-box library) for this sub-step. For ≥ 3 values of θ across orders of magnitude (e.g. 10^{-3} to 10^{2}; call `kriging_fit` with `nugget=0`, since its default is 10^{-10}), report $\kappa(\mathbf{R})$ and comment on what a large value implies for computing $\mathbf{R}^{-1}$; if $\mathbf{R}$ is (nearly) singular, apply a nugget $\mathbf{R} + \epsilon\mathbf{I}$ ($\epsilon \approx 10^{-6}$) and show it helps. Then fit Kriging on the *full* training set with a GP library (e.g. `GaussianProcessRegressor` with `normalize_y=True` and an anisotropic `RBF(length_scale=np.ones(6))` kernel, or `fitrgp` with `'BasisFunction','constant'`); report test RMSE/R^2 and the predicted std at the test points nearest to and farthest from the training set, and comment on how the uncertainty relates to distance from the training data.

⋆ (Bonus – optional, no penalty for skipping) **Comparing surrogate families.** Collect your tuned models (CV-selected; Kriging by maximum likelihood) from questions 3–6 – quadratic+ridge, RBF, ANN, Kriging – into one table of test RMSE/R^2 for `CL`, each family's complexity knob, and whether it gives native uncertainty quantification (mirroring lecture's "Family / Complexity knob / Native UQ?" comparison). State which family you'd recommend and why, citing a property from the table.

About MATLAB

Use the `help` option in MATLAB to find out about available MATLAB functions. The `help fitrgp` command will give information on the Kriging/Gaussian-process regression function and what its inputs and outputs are; similarly, `help fitrnet` for neural-network regression and `help cvpartition` for cross-validation partitioning.

About Python

Use the `help` option in Python to find out about available Python functions. The following commands will give information on some of the functions you may use:

```
help sklearn.gaussian_process.GaussianProcessRegressor
help sklearn.neural_network.MLPRegressor
help sklearn.linear_model.Ridge
help sklearn.model_selection.KFold
help scipy.interpolate.RBFInterpolator
```